

Report saved to the target repo and PR opened. Here are the required chat sections, copied from the saved report.

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

High Risk

Driven by cross-layer business-logic duplication (H4/H5) and one oversized single-function page component (H3, 213 LOC); complexity and size are otherwise moderate.

HOTSPOT SCORE (WEIGHTED COMPOSITE)

42 / 100 — Moderate

Hotspot Score = (Cyclomatic Complexity 55 × 25%) + (Code Churn 20 × 25%) + (Defect Density 25 × 20%) + (Class/Function Size 70 × 15%) + (Business Logic Duplication 72 × 10%) + (Developer Ownership Risk 15 × 5%) = 13.8 + 5.0 + 5.0 + 10.5 + 7.2 + 0.8 = 42

EXECUTIVE SUMMARY

The Klearcom codebase is small (~50 application source files across three layers) and no file breaches the 1000-LOC "large class" threshold, so raw size is healthy. The dominant problem is **duplication**: the same business rules — toll-free reachability success-rate math, the recursive IVR `buildTree` algorithm, and dashboard KPI aggregation — are reimplemented across the PHP backend, the Node `dev-api`, and the React frontend, and the two page components (`ConnectPage`, `DiscoveryPage`) are near-identical clones. Complexity is moderate (peak cyclomatic ~15 in the reachability test runners), driven by long ternary chains rather than deep nesting. One frontend component (`ConnectPage.tsx`, 213 LOC in a single function) exceeds the 200-LOC function limit. Coverage spans **Backend (PHP/Laravel, ~27 files)**, **Frontend (React/TS, ~17 files)**, and **dev-api (Node, 6 files)**. Git history is too shallow for churn analysis — only **3 commits by a single author** exist on `main`, so churn/defect/ownership metrics (H6–H8) are reported with that caveat and cannot be trended.

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	~15 (<code>runConnectTest</code>)	M
H2	Large Classes	Largest class/file LOC	<300	300–1000	>1000	276 (<code>dev-api/src/server.js</code>)	G
H3	Large Functions	Largest function LOC	<50	50–200	>200	213 (<code>ConnectPage.tsx</code>)	H
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	~14% (manual est.)	H

H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	~15% (manual est.)	H
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	<5 (3 commits total)	G
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	≤2	G
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	100% (single author)	G
H9	Resource-Cleanup / Lifecycle Leak (additional)	Timers/subscriptions with no teardown	0	1–2	>2	1 (LegacyMonitorPoller.jsx)	M
H10	Magic Numbers / Hardcoded Thresholds (additional)	Unexplained numeric literals in business logic	<5	5–15	>15	~12 sites	M

† H6–H8 caveat: `main` has only 3 commits (2026-06-10 → 2026-06-11) by a single author (`ksabai-gl`). Ratings reflect that shallow history — high ownership reads as "Good" per the KPI, but the single-contributor bus-factor is a latent risk, not a strength.

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	55	13.8
Code Churn	25%	20	5.0
Defect Density	20%	25	5.0
Class/Function Size	15%	70	10.5
Business Logic Duplication	10%	72	7.2
Developer Ownership Risk	5%	15	0.8
Hotspot Score	100%		42 / 100

2.5 Actions Required

Hotspot	Action	Rating	Priority
H4 Business Logic Duplication	Consolidate reachability rule, <code>buildTree</code> , and KPI math into shared services/utils (one source per runtime)	HIGH RISK	CRITICAL

H5 Duplicate Code (general)	Extract <code><TranscriptPanel></code> / <code><CardTable></code> and a base-controller transcript responder; add <code>jscpd</code> / <code>phpcpd</code> to CI	HIGH RISK	CRITICAL
H3 Large Functions	Split <code>ConnectPage</code> / <code>DiscoveryPage</code> into data hooks + presentational subcomponents (<80 LOC each)	HIGH RISK	HIGH
H1 High Cyclomatic Complexity	Extract step-runner loop + <code>ReachabilityResult</code> mapping to flatten ternary chains in the test runners	MODERATE	MEDIUM
H9 Resource-Cleanup Leak	Add <code>componentWillUnmount</code> / <code>useEffect</code> cleanup for <code>LegacyMonitorPoller</code> ; ensure SSE <code>reset()</code> on unmount	MODERATE	MEDIUM
H10 Magic Numbers	Extract <code>REACHABILITY_ALERT_THRESHOLD</code> and replace fabricated KPI literals with named constants	MODERATE	LOW

2.6 Expected Outcomes

- **Single source of truth for business rules** — changing the reachability threshold or IVR tree contract becomes a one-line edit instead of a 3-place, 2-language hunt, eliminating silent backend/dev-api divergence.
- **Testable units** — extracting hooks/services and a `ReachabilityCalculator` lets the currently-untested reachability and KPI math be unit-tested deterministically (removing the `random_int` flakiness noted in the audit).
- **Faster, safer reviews** — page components drop from 200+ LOC to composition-only files, so UI changes touch small, focused subcomponents.
- **Regression guard** — adding `jscpd` / `phpcpd` to CI prevents new copy-paste debt from re-accumulating.
- **Fewer production leaks** — enforcing lifecycle cleanup stops orphaned timers/streams from degrading long-running sessions.

Saved: `docs/discovery/02-code-quality-complexity.md` on the FSDKC repo (branch `discovery/code-quality-complexity-20260723`), delivered via **PR #6** → <https://github.com/shendeshweta/FSDKC/pull/6>. The orchestration UI will convert it to `docs/discovery/02-code-quality-complexity.pdf`.